

Contents

C Sharp Snippets##

Loops

All the different kind of Loops. If-Function Loop

```

1  public static void CounterLoop(int num1, int num2)
2  {
3      if (num1 < num2)
4      {
5          Console.WriteLine(num1);
6          CounterLoop(num1 + 1, num2);
7      }
8  }
9
10 static void Main(string[] args)
11 {
12     int number1 = 0;
13     int number2 = 30;
14     CounterLoop(number1, number2);
15 }
```

Data Structures

Binary Tree

```

1          Root
2         /  \
3        //   \
4       Child1 Child2
5        /  \   /  \
6       GC1 GC2 GC3 GC4
```

Mastering BST Removal: The “Easy” Cases (0 or 1 Child)

When removing a node from a Binary Search Tree, you always check its children first. If the node has zero children or only one child, we call this an “Easy Case.”

Why is it easy? Because you don’t need to find a Successor. You simply “bypass” the node you want to delete and reconnect the tree.

Case 0: The Leaf Node (Zero Children)

A leaf node sits at the very bottom of the tree. It has no left child and no right child.

The Initial Tree

Imagine we want to remove [20].

```

1          [50] <-- Parent
2         /    \
```

```

3    ---> [20]    [70]
4      /      \      \
5    (null) (null) [80]

```

The Logic Steps

Identify Target: We found [20].

Check Children: Left is null. Right is null. It's a leaf!

Determine Replacement: Since there are no children to save, the "replacement" for [20] is simply null.

The Snip: We tell [50] (the Parent): "Whatever hand was holding [20], let go and hold null instead."

The Resulting Tree

The node [20] is dropped, and garbage collection will clean it up in memory.

```

1      [50]
2     /  \
3    (null) [70]
4           \
5          [80]

```

Case 1: The Single Child Node

A node with one child acts like a single link in a chain. If you remove that link, you just connect the two pieces on either side of it together.

The Initial Tree

Imagine we want to remove [70].

```

1      [50] <-- Parent
2     /  \
3    [20] [70] <--- Target to delete
4           \
5          [80] <-- The only child

```

The Logic Steps

Identify Target: We found [70].

Check Children: Left is null. Right is [80]. It has exactly one child!

Determine Replacement: The replacement for [70] is its only child, [80].

The Bypass: We tell [50] (the Parent): "Whatever hand was holding [70], let go and grab [80] instead."

The Resulting Tree

The node [70] is entirely bypassed. [50] now connects directly to [80]. The tree structure remains perfectly valid!



The “Root” Exception

There is one special scenario in both of these cases. What if the node you are deleting is the very top of the tree (the Root)?

If you delete the Root, there is no Parent. (Parent == null).

If the Root is a Leaf (Case 0): The whole tree becomes null.

If the Root has One Child (Case 1): That single child gets promoted to become the brand new Root of the entire tree.

The Code Logic Concept

In both Case 0 and Case 1, your code only needs to ask two questions:

What is the Replacement? (Answer: Whichever child is NOT null. If both are null, the replacement is null).

Where does the Replacement go? (Answer: Attach it to the Parent. If there is no Parent, it becomes the new Root).

0.0.0.1 Removal

- Removal in Binary Search Tree involves finding the correct node for removal. Storing the value

Mastering BST Removal: The “Two-Child” Case

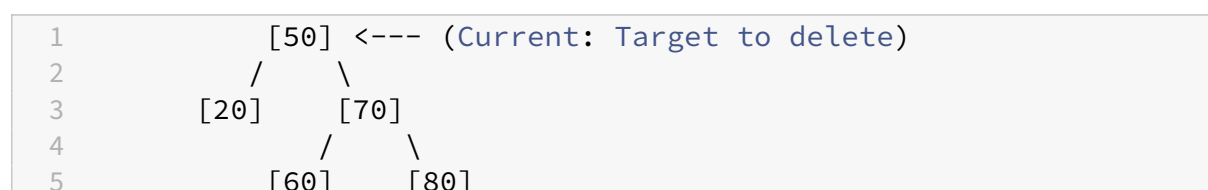
When you remove a node with zero children (a leaf) or one child, it’s easy: you just cut the string and tie it to the next thing down.

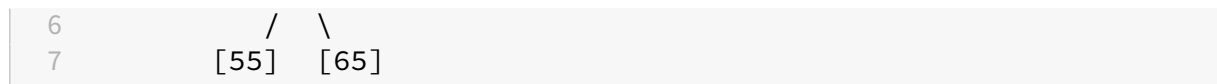
But when a node has two children, you can’t just delete it. If you delete the root, what happens to the left and right branches? They fall apart.

To fix this, we use a strategy called “The Stand-In” (or finding the In-Order Successor).

The Initial Tree

Let’s imagine a tree where we want to remove the Root (50).





Why is this hard?

If we just delete [50], we have two floating subtrees ([20] and [70]). We need a New King to take the [50] spot.

The Rule: The New King MUST be greater than everything on the left ([20]), and less than everything on the right ([70], [60], etc.).

Step 1: Find the Successor (The “New King”)

The only logical replacement is the smallest number in the right branch. How do we find it logically in code?

Take one step to the Right.

Walk as far Left as possible until you hit a dead end.

Let’s trace it:

Go Right from [50] -> You are at [70].

Go Left -> You are at [60].

Go Left -> You are at [55].

Try to go Left -> It’s null. You found the Successor!

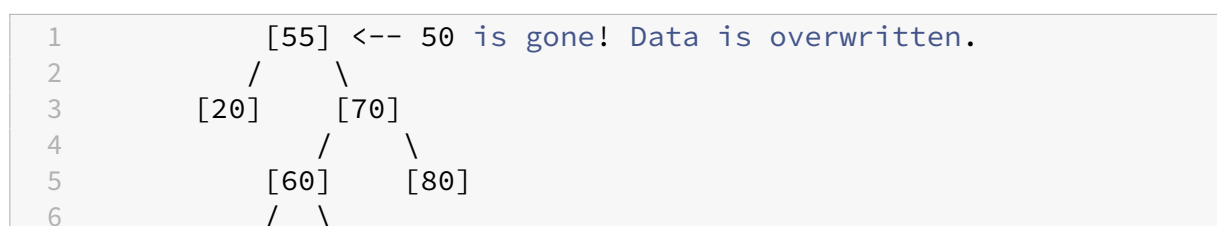


In your code logic: You create a successor variable that points to [55], and a successorParent variable that points to [60].

Step 2: The “Identity Theft” (Swap the Data)

Instead of moving the actual [55] node object all the way to the top (which means rewriting tons of Left and Right pointers), we just copy the data.

We overwrite the Target’s Key and Value with the Successor’s Key and Value.



```
7      [55] [65] <-- Wait, we have two 55s now!
```

In your code logic: `Current.Key = successor.Key;`

Step 3: The Cleanup (Snip the old node)

We successfully updated the top of the tree, but we left the old “Stand-In” node at the bottom. We must delete it.

The Golden Rule of the Successor: Because we went as far Left as possible to find it, the Successor is guaranteed to have NO Left child. It might have a Right child, or no children at all.

This means deleting the old successor is an “Easy Case” removal!

We tell the `successorParent` ([60]) to bypass [55] and point to whatever is on [55]’s right side (in this case, null).

```
1      [55]
2     /  \
3    [20]  [70]
4       /  \
5      [60]  [80]
6     /  \
7    (null) [65] <-- The old 55 is snipped away!
```

In your code logic: Since the old [55] was the Left child of [60], we do: `successorParent.Left = successor.Right;`

Summary of the Logic

Target has two children? Find the Successor.

Successor = Right once, then Left-Left-Left.

Overwrite Target’s Data with Successor’s Data.

Tell Successor’s Parent to bypass the Successor.

##Sorting Algorithms##

###Quick Sort###

```
1 class Program
2 {
3     // These are our 'Global' evidence storage areas
4     static int[] Keys = { 105, 101, 103, 102, 104 };
5     static int[][] database = {
6         new int[] { 5, 5, 5 },
7         new int[] { 1, 1, 1 },
8         new int[] { 3, 3, 3 },
9         new int[] { 2, 2, 2 },
10        new int[] { 4, 4, 4 }
11    };
```

```
12
13 // THE HANDS: Physically moves the evidence
14 static void Swap(int a, int b)
15 {
16 // Swap the label (Key)
17 int tempKey = Keys[a];
18 Keys[a] = Keys[b];
19 Keys[b] = tempKey;
20
21 // CRITICAL: Swap the actual evidence drawer (Database row)
22 // This ensures the data stays 'glued' to its key
23 int[] tempRow = database[a];
24 database[a] = database[b];
25 database[b] = tempRow;
26 }
27
28 // THE ENGINE: Organizes one section of the array
29 static int Partition(int low, int high)
30 {
31 // Pick the last item as the 'Benchmark' (Pivot)
32 int pivot = Keys[high];
33
34 // 'i' marks the 'Wall' of the Small Group.
35 // It starts off-stage because we haven't checked anyone yet.
36 int i = low - 1;
37
38 // 'j' is the Scout. It walks from the start of the section to
   the end.
39 for (int j = low; j < high; j++)
40 {
41 // If the Scout finds a key smaller than our Benchmark...
42 if (Keys[j] < pivot)
43 {
44 // Move the wall to make a seat, then swap the item into that
   seat
45 i++;
46 Swap(i, j);
47 }
48 }
49
50 // After the loop, put the Benchmark (Pivot) right after the
   wall.
51 // It is now in its final, permanent home.
52 Swap(i + 1, high);
53
54 // Tell the manager where the Pivot is sitting
55 return i + 1;
56 }
57
58 // THE MANAGER: Coordinates the 'Divide and Conquer' strategy
59 static void QuickSort(int low, int high)
```

```
60 {
61 // Base Case: Only sort if there is more than one item in the
    section
62 if (low < high)
63 {
64 // 1. Run the engine to find the 'Middle Point' (pi)
65 int pi = Partition(low, high);
66
67 // 2. RECURSION: Sort the Left side (everything smaller than
    the middle)
68 QuickSort(low, pi - 1);
69
70 // 3. RECURSION: Sort the Right side (everything larger than
    the middle)
71 QuickSort(pi + 1, high);
72 }
73 }
74
75 // THE REPORT: Displays the current state of the database
76 static void PrintDatabase()
77 {
78 for (int i = 0; i < Keys.Length; i++)
79 {
80 // Print the key first
81 Console.Write($"Keys: {Keys[i]} | Data: ");
82
83 // Open the specific drawer and print each number inside
84 for(int j = 0; j < database[i].Length; j++)
85 {
86 Console.Write(database[i][j] + " ");
87 }
88 // New line after finishing each row
89 Console.WriteLine();
90 }
91 }
92
93 static void Main(string[] args)
94 {
95 Console.WriteLine("---Evidence Before Sorting");
96 PrintDatabase();
97
98 // Start the sort: from the first index (0) to the last index (
    Length - 1)
99 QuickSort(0, Keys.Length - 1);
100
101 Console.WriteLine("\n --- Evidence After Sorting ---");
102 PrintDatabase();
103
104 Console.WriteLine("\n Print any key to exit ");
105 Console.ReadKey();
106 }
```

```
107 }
```

##Files and Data##

File Write and Load

```
1  class Program
2  {
3      static private int[]? loc;
4      static private string[][]? array;
5      static private int Count;
6      static void textWrite(string text1, string text2)
7
8
9      {
10
11         //name of the file
12         string filename = "data.csv";
13         //Sub directory
14         string path1 = @"data/";
15
16         //Getting full path
17         string fullpath = System.IO.Path.Combine(System.IO.Path.
            GetFullPath(path1), filename);
18
19         //Declaring writer object
20         System.IO.StreamWriter writer;
21         //Initializing
22         writer = new System.IO.StreamWriter(fullpath);
23
24
25         writer.WriteLine(text1);
26         writer.WriteLine(text2);
27         //Flush() and close() file to save memory.
28         Console.WriteLine("Success");
29         //Close and flush writer so data can be transferred from buffer
            to the file.
30         writer.Close();
31
32
33     }
34
35
36     static void arrayWrite()
37     {
38         Count = 5;
39         loc = new int[]{0, 1, 2, 3, 4};
40         array = new string[Count][];
41         array[0] = ["Ajmal", "Emal", "Ahmad", "Qomandan", "Toofan"];
42         array[1] = ["Ben", "mark", "ahmad", "megan", "lala", "mikhael"
            ];
43         array[2] = ["January", "February", "March", "April", "May", "
```

```
        "June", "July", "August", "September", "October", "November",  
        "December"];  
44 array[3] = ["Sunday", "Monday", "Tuesday", "Wednesday", "  
        Thursday", "Friday", "Saturday"];  
45 array[4] = ["Summer", "Winter", "Autumn", "Spring"];  
46  
47  
48 //name of the file  
49 string filename = "data.csv";  
50  
51 //Sub directory  
52 string path1 = @"data/";  
53  
54 //Getting full path  
55 string fullpath = System.IO.Path.Combine(System.IO.Path.  
        GetFullPath(path1), filename);  
56  
57 //Declaring writer object  
58 System.IO.StreamWriter writer;  
59 //Initializing  
60 writer = new System.IO.StreamWriter(fullpath);  
61  
62 for(int i = 0; i < loc.Length; i++)  
63 {  
64     writer.Write($"{loc[i]}"); Console.WriteLine($"Writing: LOC. {  
        loc[i]}");  
65     for (int j = 0; j < array[i].Length; j++)  
66     {  
67         writer.Write($"",{array[i][j]}");  
68     }  
69     writer.Write("\n");  
70 }  
71 writer.Close();  
72 }  
73  
74 static int[] ConvertToInt(string[] parts){  
75  
76     int[] Parts = new int[parts.Length];  
77  
78     for (int i = 0; i < parts.Length; i++)  
79     {  
80         Parts[i] = Convert.ToInt32(parts[i]);  
81     }  
82  
83     return Parts;  
84 }  
85 }  
86  
87 static void LoadFile(string file){  
88  
89     string line = null;
```

```
90  int lineCounter = 0;
91  int[] LoadKeys = new int[100];
92  string[][] LoadDatabase = new string[100][];
93
94  string filePath = $"./data/{file}";
95  System.IO.StreamReader reader;
96  reader = new System.IO.StreamReader(filePath);
97
98
99  while ((line = reader.ReadLine()) != null){
100
101  string[] Parts = line.Split(",");
102  int[] PartsConvertedToInt = ConvertToInt(Parts);
103  LoadKeys[lineCounter] = PartsConvertedToInt[0];
104  LoadDatabase[lineCounter] = new string[Parts.Length - 1];
105
106  for(int i = 1; i < Parts.Length; i++){
107  LoadDatabase[lineCounter][i - 1] = PartsConvertedToInt[i];
108  }
109  lineCounter++;
110
111  }
112  loc = LoadKeys;
113  array = LoadDatabase;
114  reader.Close();
115
116  for(int i = 0; i < lineCounter; i++)
117  {
118  Console.Write($"{loc[i]}");
119  for (int j = 0; j < array[i].Length; j++)
120  {
121  Console.Write($",{array[i][j]}");
122  }
123  Console.WriteLine("\n");
124  }
125
126  }
127  static void Main(string[] args)
128  {
129  //Console.WriteLine("Text1: ");
130  //string Input1 = Console.ReadLine();
131  //string Input2 = Console.ReadLine();
132
133  //textWrite(Input1, Input2);
134  //arrayWrite();
135  string FileName = Console.ReadLine();
136  LoadFile(FileName);
137
138
139  // to load arrayWrite();
140  }
```

```
141  
142 }
```

##All Other Projects##

###Minimum of Two###

```
1 namespace MinimumOfTwo;  
2 //This Program takes two inputs of numbers and places them in  
   separte arrays. It compares array1 index to array2 index to  
   find the larger number. The larger number is placed into a  
   temporary array which is returned.  
3 class Program  
4 {  
5     static int[] CompareTwoArrays(string[]? array1, string[]?  
        array2)  
6     {  
7         int GetLength = array1.Length > array2.Length ? array1.Length :  
            array2.Length;  
8         int[] result = new int[GetLength];  
9  
10        for (int i = 0; i < GetLength; i++)  
11        {  
12  
13            if (Int32.Parse(array1[i]) > Int32.Parse(array2[i]))  
14            {  
15                result[i] = Int32.Parse(array1[i]);  
16  
17            }  
18            else  
19            {  
20                result[i] = Int32.Parse(array2[i]);  
21  
22            }  
23  
24        }  
25        return result;  
26    }  
27  
28  
29    static void Main(string[] args)  
30    {  
31        string[] delimiter = new string[] { " ", "," };  
32        Console.Write("Enter Input1: ");  
33        string[]? input1 = Console.ReadLine().Split(delimiter,  
            StringSplitOptions.None);  
34        Console.Write("Enter Input2: ");  
35        string[]? input2 = Console.ReadLine().Split(delimiter,  
            StringSplitOptions.None);  
36  
37        int[] result = CompareTwoArrays(input1, input2);  
38        Console.WriteLine("Results: ");
```

```
39  for (int i = 0; i < result.Length; i++)  
40  {  
41  
42    Console.Write(result[i] + " ");  
43  
44  }  
45  
46  
47  }  
48 }
```